

Integrating Symbolic Execution with LLMs for Automated Generation of Program Specifications

Fanpeng Yang, Xu Ma, Shuling Wang*, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu

Abstract—Automatically generating formal specifications including loop invariants, preconditions, and postconditions for legacy code is critical for program understanding, reuse and verification. However, the inherent complexity of control and data structures in programs makes this task particularly challenging. This paper presents a novel framework that integrates symbolic execution with large language models (LLMs) to automatically synthesize formally verified program specifications. Our method first employs symbolic execution to derive precise strongest postconditions for loop-free code segments. These symbolic execution results, along with automatically generated invariant templates, then guide the LLM to propose and iteratively refine loop invariants until a correct specification is obtained. The template-guided generation process robustly combines symbolic inference with LLM reasoning, significantly reducing hallucinations and syntactic errors by structurally constraining the LLM’s output space. Furthermore, our approach can produce strong specifications without relying on externally provided verification goals, enabled by the rich semantic context supplied by symbolic execution, overcoming a key limitation of prior goal-dependent tools. Extensive evaluation shows that our tool SESpec outperforms the existing state-of-the-art tools across numerical and data-structure benchmarks, demonstrating both high precision and broad applicability.

Index Terms—Program specification generation, loop invariants, symbolic execution, large language models, formal verification.

I. INTRODUCTION

Across diverse software domains from general-purpose systems to safety-critical fields such as aerospace, a vast body of legacy code lacks formal specifications that precisely define its functional behavior. Automating specification generation is therefore essential for program comprehension, reuse, and verification. However, this task remains profoundly challenging due to the intricate interplay of control structures (e.g., nested branches, loops, function calls) and complex data representations (e.g., structs, pointers, inductive datatypes). In this work, we tackle the problem of functional correctness specification generation, which can be viewed as functional summarization: inferring specifications that are as strong and precise as

possible to fully characterize program behavior, rather than producing specifications tailored to specific, pre-defined verification goals. Crucially, such specifications are not synthesized with respect to any particular verification goal, yet their logical strength makes them directly applicable to a wide range of verification tasks.

Existing research on program specification generation has largely focused on loop invariants. Traditional approaches such as constraint solving [1], [2], Craig interpolation [3]–[5], recurrence analysis [6], [7], shape analysis [8], *etc.*, are each restricted to specific invariant forms or program classes, and their complexity often hinders scalable, automated inference, requiring manual guidance. Dynamic methods like Daikon [9] synthesize invariants from test executions against templates, but offer no formal guarantees and are sensitive to test quality. Recent efforts combine machine learning with SMT solvers to improve efficiency [10]–[12], but these methods remain largely confined to numeric programs. More recently, Large Language Models (LLMs) have been applied to the specification generation for broader programs. While LLMs exhibit impressive semantic understanding, they struggle to generate specifications that are both syntactically correct and semantically valid: hallucinations may lead to ill-formed specifications, and even well-formed ones tend to be too weak or incorrect to faithfully capture program behavior.

Several recent efforts integrate LLMs with static and formal techniques [13]–[16]. AutoSpec [13], a state-of-the-art LLM-based approach, decomposes programs via static analysis and synthesizes specifications bottom-up. However, its generation remains fundamentally LLM-driven, making it susceptible to both syntactic and semantic hallucinations. It is also goal-directed, requiring predefined verification targets, and its performance degrades for complex data structures. Preguss [16] addresses a different problem: specification synthesis for runtime error (RTE) elimination rather than functional correctness. It employs abstract interpretation to generate RTE assertions and synthesizes minimal contracts sufficient to discharge them. Its goal is safety guarding, not functional summarization. Consequently, Preguss requires externally supplied RTE targets, cannot operate without them, and it also explicitly struggles with inductive data structures such as linked lists. Both approaches are fundamentally goal-directed: their specifications are often tailored to a predefined verification target and may over-approximate behaviors outside it, thus failing to describe a program’s overall functional correctness. In real-world engineering, however, the vast majority of legacy code lacks any such explicit goals, yet the need for precise functional specifications remains critical.

*Corresponding author: Shuling Wang (e-mail: wangsl@ios.ac.cn).

F. Yang, X. Ma, S. Wang, and X. Xu are with the Institute of Software, Chinese Academy of Sciences, and also with the University of Chinese Academy of Sciences, Beijing, China (e-mail: yangfp@ios.ac.cn; maxu@ios.ac.cn; wangsl@ios.ac.cn; xux@ios.ac.cn).

Q. Cao is with the School of Computer Science and School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China (e-mail: cao-qinxiang@sjtu.edu.cn).

N. Zhan is with the School of Computer Science, Peking University, Beijing, China (e-mail: znj@ios.ac.cn).

X. Li and B. Gu are with the Beijing Institute of Control Engineering, Beijing, China (e-mail: 1351127502@126.com; gubin@ios.ac.cn).

To overcome these limitations, we propose SESpec, a novel framework that generates formally verified functional specifications without requiring any explicit verification goal. Our key insight is to tightly integrate symbolic execution, LLMs, and formal verification into a unified framework, which handles diverse program constructs in a principled manner. For loop-free code, symbolic execution directly computes strongest postconditions, yielding exact input-output functional summaries. For loops, path-sensitive symbolic results and equality-based invariant templates guide the LLM to propose and iteratively refine invariants via verification feedback. We handle complex constructs compositionally. Function calls and nested loops are managed using stacks that enable compositional specification generation from inner to outer scopes. Non-recursive data structures, including pointers, arrays, and structs, are modeled using a flattened memory representation that explicitly tracks separation and aliasing, while recursive data structures are specified using inductive predicates. Together, these techniques are designed to collectively infer maximally precise functional correctness specifications for a diverse scope of programs. In the actual implementation, we leverage QCP’s symbolic execution engine [17], [18] for its ability to precisely track symbolic execution results at arbitrary execution points, combined with Frama-C [19] as our formal verifier due to its automated verification capability and also ACSL-style assertions that are amenable to LLM processing. Our framework tightly integrates QCP, Frama-C and LLMs, where each component’s strengths compensate for the others’ limitations.

We performed a comprehensive and rigorous experimental evaluation to validate our approach. This includes an ablation study confirming the contribution of each technical component, along with extensive benchmark assessments. The results demonstrate the effectiveness and broad applicability of our approach across diverse tasks in invariant and program specification generation. On numerical benchmarks, our method performs competitively with specialized tools designed for numerical programs. Compared to recent LLM-based approaches such as AutoSpec and ACInv, our approach demonstrates superior performance on numerical benchmarks. Furthermore, we introduced three new benchmarks targeting distinct programs: more complex numerical programs, multi-layered structs and branches derived from real aerospace applications, and recursive data structures with singly linked lists. Our method significantly outperforms both AutoSpec and ACInv across these benchmarks, demonstrating the generalizability and robustness of our method across a wide spectrum of programs.

Additionally, we evaluated our method on loop invariant and program specification generation under masked verification goals using the SyGuS and Frama-C benchmarks. The results indicate that our approach continues to produce high-quality specifications with a small drop in performance while substantially outperforming the state-of-the-art tool AutoSpec under identical masking conditions. These findings highlight the robustness of our approach even in the absence of explicit verification goal guidance.

In summary, the key contributions of this work are:

- **Symbolic execution-guided specification synthesis:** We present the first framework that leverages precise symbolic execution to guide LLM-based specification generation, providing exact semantic context that reduces hallucinations and improves specification quality.
- **Template-based loop invariant generation:** We introduce equality-constraint templates derived from symbolic execution that structurally constrain LLM outputs, automatically incorporating critical logical constraints that pure LLM approaches consistently miss.
- **Hierarchical iterative refinement:** We design a multi-layer feedback mechanism that categorizes verifier errors and applies targeted repair strategies, effectively guiding the LLM toward verified solutions through progressive refinement.
- **Goal-independent specification generation:** Unlike goal-directed approaches, our method generates strong functional specifications even without predefined verification goals, evidenced by our experiments.
- **Comprehensive evaluation:** We demonstrate state-of-the-art performance across numerical programs, complex data structures, and programs featuring structs and intricate control flow derived from real aerospace software, significantly outperforming existing approaches.

The rest of the paper is organized as follows: Sect. II introduces a motivating example and provides an overview of our approach. Sect. III and Sect. IV detail our core technical contributions on the generation of function specifications and loop invariants. Sect. V presents the overall system design and implementation details of our approach. Sect. VI evaluates our approach empirically. Sect. VII presents three representative case studies. Sect. VIII discusses threats to validity. Finally, Sect. IX presents the related work, and Sect. X concludes the paper with directions for future work. The details on tool implementation, LLM configurations, and experimental results are available at <https://github.com/anon-hikityq/TSE2026-SESpec>.

II. A MOTIVATING EXAMPLE AND OUR APPROACH

Automating the generation of formal specifications for real-world C code remains challenging due to the prevalence of complex language features such as loops, pointer manipulations and data structures. Consider the code fragment shown in Fig. 1, which originates from the Sun Search system, a legacy embedded software component in the aerospace domain. The code starts to define a struct `CheckCal` with three fields: an integer array `pkv[10]`, an integer field `len` representing the number of valid elements in the array, and an integer `chksum` storing the sum of the array’s elements. Then, function `foo` invokes `CheckCalFun` on a `CheckCal` pointer `pIp`, with a given verification goal annotated by `/*@assert ... */`. The callee function `CheckCalFun` takes as input a `CheckCal` pointer `pIp`, computes the sum of the `len` elements in the array `pkv` via a loop and then stores the result in the field `chksum` of `pIp`. This example exhibits diverse features: complex data structures (structs, arrays and pointers), a loop with inductive operations over arrays, and

pointer-induced aliasing and side effects. In consequence, it faces the following challenges:

- **Loop reasoning:** The summation loop `for(; i < pIp->len; i++)` requires determining whether the initial condition allows to enter the loop, and if the loop executes, establishing the invariants sufficient enough to specify the precise behavior of the loop, i.e. to relate the partial sum to the processed prefix of the array.
- **Structs and Pointers:** The procedures involve operations such as accessing arrays, reading and writing to struct fields via pointers, making it essential to account for the resulting aliasing and side effects in their specifications. For instance, the specification for function `CheckCalFun` must guarantee memory safety for dereferencing `pIp->pkv[i]`, precisely describe the write effect on `pIp->chksum`, and specify the frame property for other fields of `pIp` that remain unchanged.
- **Inductive predicates:** Properties of arrays in loop such as aggregate sums are most naturally expressed with inductive predicates. Automatically writing correct forms of such predicates and furthermore verifying them remains a central challenge for programs involving recursive operations or data structures.

As shown in Fig. 1, the code is annotated with the full ACSL specifications enclosed in `/* @... */`, including preconditions (beginning with `requires`), postconditions (beginning with `ensures`), loop invariants (beginning with `loop invariant`), and verification goals (beginning with `assert`). Among these, the loop invariants, the default preconditions and the postconditions are generated automatically by our tool `SESpec`, while the `assert` clause represents verification goals provided a priori, and certain (non-default, line 4-5) preconditions reflect additional requirements from the designers.

Before presenting our method, we attempted to generate the specification for this example using GPT-4o, GPT-5 and AutoSpec respectively, given both the preconditions and the verification goal. The loop invariants they produced, shown below, were all found to be incorrect:

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(0, i-1, \lambda integer k;
    pIp->pkv[k]);          (GPT-4o)

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(integer k = 0; k < i; pIp->pkv[k]);
loop assigns i, chksum;
loop variant pIp->len - i;    (GPT-5)

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(0, i-1, pIp->pkv);
loop invariant \forall integer k;
  0 <= k < i ==> chksum >= pIp->pkv[k];
loop assigns i, chksum;      (AutoSpec)

```

The failures highlight the common limitations of pure LLM-based approaches. First, LLMs often neglect whether a loop is ever entered, so their generated invariants presume loop entry without establishing necessary preconditions (e.g., `pIp->len` might be negative). Even when we explicitly prompted GPT-4o to "consider the case when the loop is not entered", it still failed to generate the correct conditional invariant structure. Second, when reasoning about arrays, LLMs introduce uninterpreted functions (e.g., a custom `sum`) that are undefined in the target verification framework (such as `Frama-C`), causing verification failures.

A. Our Approach

Fig. 2 shows an overview of our approach. Without loss of generality, given a function and an optional verification goal, the objective is to automatically generate loop invariants and function pre-/postconditions. If a verification goal is provided, these specifications must be sufficient to prove it; otherwise, they should be strong enough to capture the function's core behavior.

The entire process **Function Specification Generation** comprises four stages, as illustrated in Fig. 2. It begins with **Default Precondition Generation** (marked in green), which produces the weakest preconditions ensuring memory-safe execution (node ① in Fig. 1). Our tool employs a flattened memory model that recursively decomposes input data structures into atomic memory locations, generating validity and separation constraints to guarantee valid memory accesses. Since other properties beyond memory safety are not required for symbolic execution, they are left for users to specify as needed.

Using this precondition, the tool initiates symbolic execution from the function's entry point (node ① in Fig. 1). When a function call is encountered (node ② in Fig. 1), the current symbolic execution is paused and the tool call **Callee Specification Generation** (mark in purple in Fig. 2). It recursively processes the callee function to generate its specification (node ③ - node ⑥ in Fig. 1). Upon completion, the tool returns to the call site, incorporates the generated pre-/postcondition of the callee and resumes symbolic execution of the caller. Throughout this process, a call stack is maintained to support nested function calls. Functions are processed in a bottom-up order, and all generated function specifications are stored for reuse across different calling contexts.

When symbolic execution encounters a loop within the current function (node ⑤ in Fig. 1), it pauses and transfers control to the **Loop Invariant Generation** (mark in blue in Fig. 2), the core component of our tool. The Loop Analysis module is first invoked, taking as input the complete symbolic state at the loop entry, and identifies different types of variables by analyzing the behavior of the loop body. In the presented example, the analysis produces the following results: the unchanged variables `pIp->len`, `pIp` and `pIp->pkv`; the inductive variables `i` and `chksum`. Additionally, the loop entry condition is established, i.e., `i < pIp->len`. Subsequently, the tool generates parameterized invariant templates designed to capture various execution scenarios, such as whether the

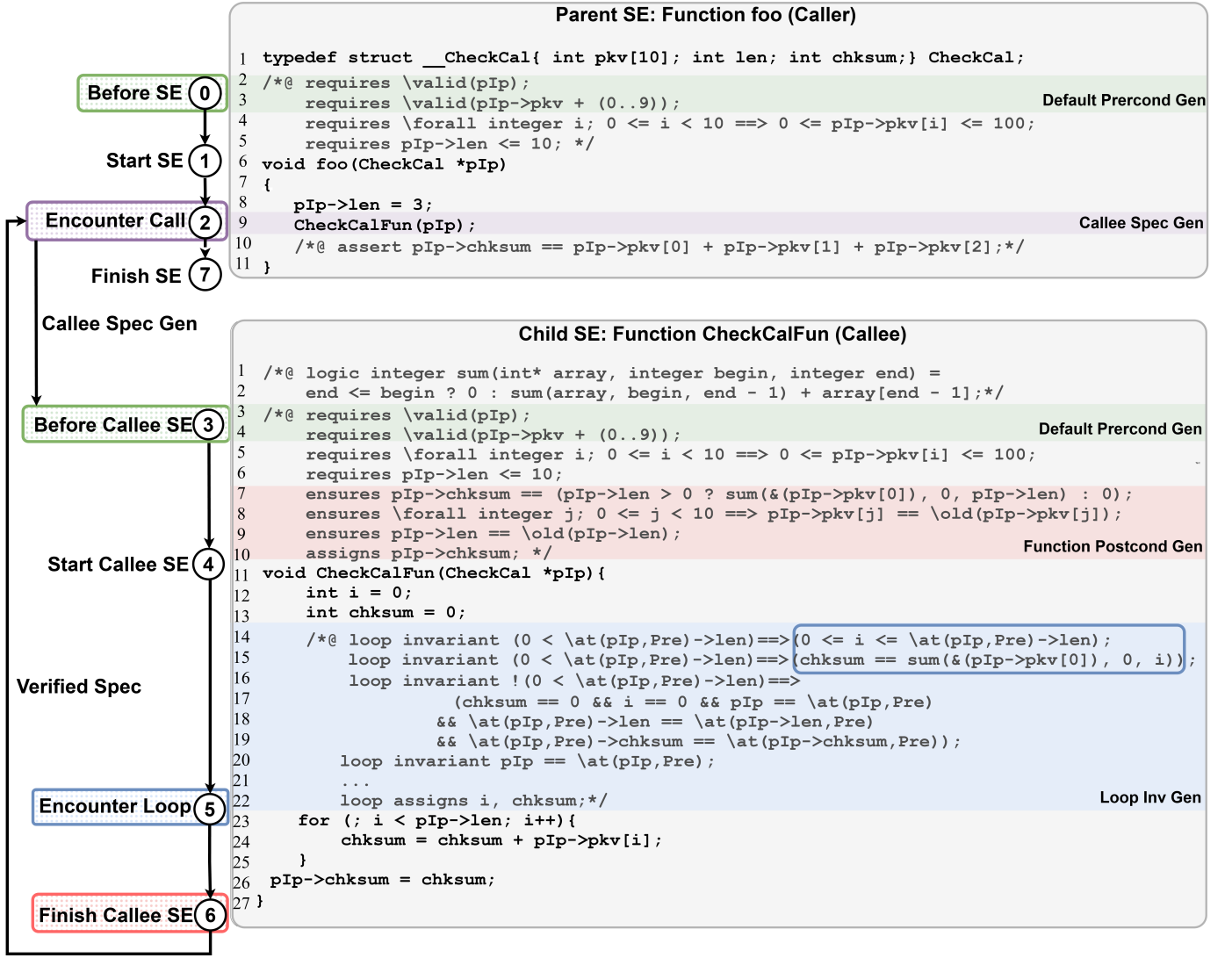


Fig. 1. Motivating Example

loop is entered, which variables remain unchanged, which evolve inductively and how they behave. These templates are expressed as ACSL skeletons using equality constraints to facilitate the synthesis of more precise invariants. Below we present part of the generated templates for the motivation example (see Fig. 1 for the full set):

```

loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE HOLDER_for_i;
loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE HOLDER_for_chksum;
loop invariant pIp == \at(pIp, Pre);

```

These skeletons provide specific PLACE HOLDER tags for the LLM to populate (marked as the blue boxes in Fig. 1). Guided by these structured templates and symbolic execution results, LLM produces loop invariants that precisely characterize the behavior of different types of variables. This structural guidance significantly reduces hallucinations, minimizes syntax errors, and narrows the overall search space. For instance, the generated invariants correctly incorporate

the loop-entry condition $(0 < \text{\texttt{\textbackslash at}(pIp, Pre)} \rightarrow \texttt{len})$, a critical logical constraint that pure LLM-based approaches consistently overlook.

During the Loop Invariant Generation, a refinement procedure is iteratively invoked to assess the validity (able to pass the formal verification) and the accuracy (sufficient to guarantee the given verification goal) of the generated loop invariants. Depending on the feedback from the verifier, different refinement strategies are applied to guide the LLM toward correct invariants.

When symbolic execution of the callee is completed (node ⑥ in Fig. 1), the **Function Postcondition Generation** (mark in red in Fig. 2) is triggered, the resulting symbolic states are used to synthesize a postcondition that summarizes the callee's functional behavior. Upon completion, the generated pre-/postconditions are returned to the call site and reused as a functional summary in the caller (node ⑦ in Fig. 1), enabling symbolic execution of the caller to resume without re-analyzing the callee's internal control flow.

If symbolic execution has not completed, it proceeds

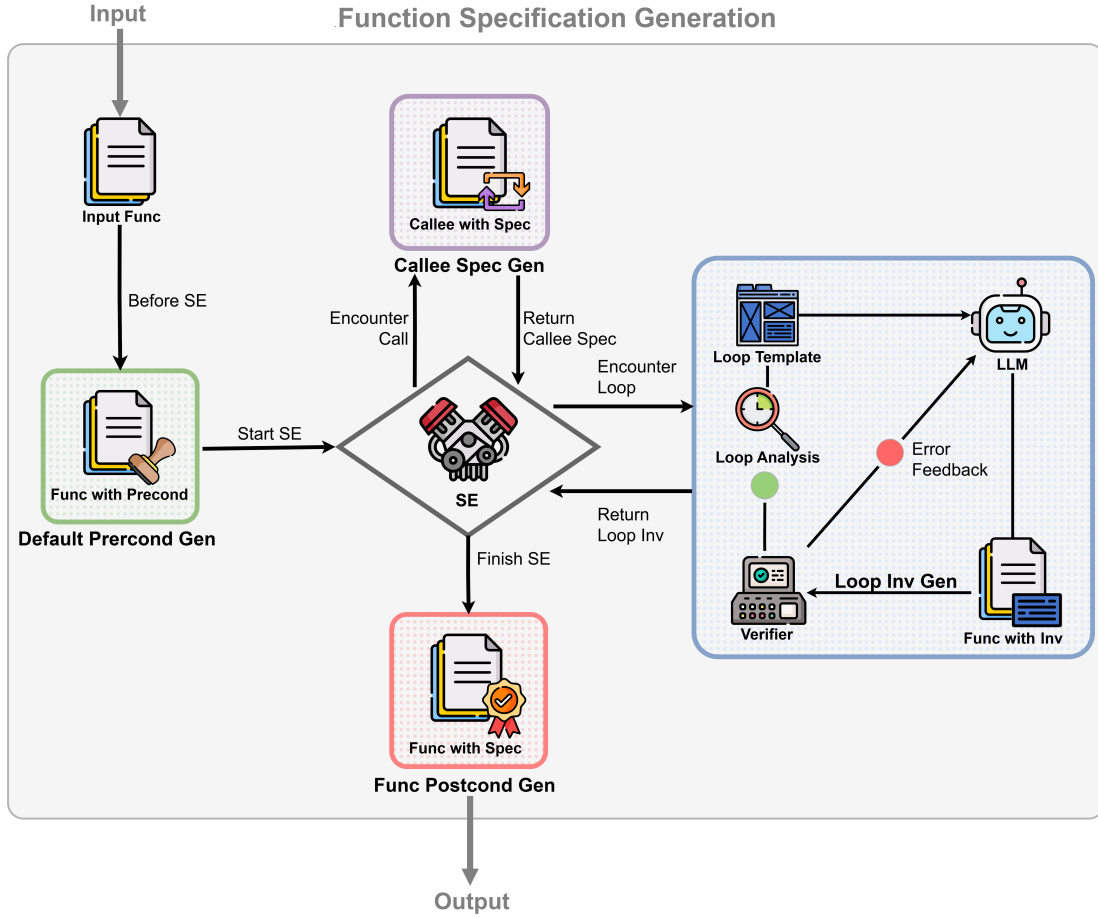


Fig. 2. Overview of Our Approach

until the final postcondition of the function is generated. Consequently, our approach synthesizes complete functional summaries for this example, including the weakest preconditions (marked in green) that ensure memory safety, the loop invariants that capture loop-entry conditions, inductive effects, and the identification of unchanged variables, and the postconditions sufficient to prove the given verification goal. Notably, the same postcondition for `CheckCalFun` can still be generated even when the verification goal is masked, demonstrating that our specification generation is not solely driven by the goal itself.

III. PRE-/POSTCONDITION SPECIFICATION GENERATION

This section first introduces the generation of strongest specifications for code segments without loops via symbolic execution, and then presents the generation of general function specifications. Loop invariant generation is deferred to next section.

A. Symbolic Execution and Strongest Pre-/Postconditions

We use a *flattened memory model* to explicitly track memory values and aliasing during symbolic execution. In this model, each program point p is associated with a scope consisting of a set of variables accessible at that point, and the memory at p is modeled as a collection of atomic memory locations $\mathcal{L}_p$

reachable from these variables, each of which is treated as a symbolic variable corresponding to a primitive memory unit.

Within this model, the symbolic execution of statements maintains a symbolic state (PC_π, σ_π) for each execution path π , where

- PC_π denotes the path condition, i.e. a logical formula over initial symbolic values of $\mathcal{L}_p$ and constants, that expresses the conditions required to traverse path π . PC_π is updated when a branch is taken along π .
- σ_π denotes the symbolic store along path π , i.e. a logical formula specifying the symbolic values of $\mathcal{L}_p$. σ_π is updated when statements such as assignments or memory updates execute along π .

The symbolic execution of a program is defined as a transformation over sets of symbolic states, accounting for divergent control flow at branches. Formally, execution begins at the function entry point p_0 with an initial symbolic state set containing a single element (PC_0, σ_0) . Here, σ_0 assigns initial symbolic values to all variables in the logical variable set $\mathcal{L}_p$, and PC_0 is the path condition that encodes the precondition constraining those initial values. For any program point p , the set of all reachable symbolic states is given by the collection of pairs $\{(PC_{\pi_i}, \sigma_{p, \pi_i})\}_{i=1}^m$ for every path π_i from the entry point p_0 to p . From this set, we derive a logical formula characterizing all feasible states at p , defined

as $\bigvee_{i=1}^m (PC_{\pi_i} \wedge \sigma_{p,\pi_i})$. At the entry point p_0 , this formula simplifies to the precondition $PC_0 \wedge \sigma_0$. For any subsequent program point, the corresponding formula represents the postcondition that holds up to that point in execution.

We implement a symbolic execution procedure $SEStart(\mathcal{F}, Pre, p_{start})$. Given a code segment $\mathcal{F}$ and a precondition Pre (a formula representing symbolic states) at a starting point p_{start} , the procedure executes symbolically from p_{start} , exploring the execution paths until it encounters a loop or a function call.

Based on the above definition of symbolic execution, we can derive the *strongest specifications* for loop-free, straight-line code. It consists of a weakest precondition and a strongest postcondition, together giving the most precise description of the input/output behavior. In our approach, a default weakest precondition is automatically generated, requiring that all accessible atomic memory locations at the entry point are initialized and contain valid symbolic values. Given a precondition Pre at the entry point p_{start} , the strongest postcondition at p_{end} for $\mathcal{F}$, denoted by $SP(\mathcal{F}, Pre, p_{start}, p_{end})$, is the logical formula describing the disjunction of all symbolic states (defined as above) reachable at p_{end} from p_{start} under Pre .

B. Generation of Program Specifications

Without loss of generality, we describe the process of generating specifications of functions based on symbolic execution by Algorithm 1. The algorithm takes as inputs a function $\mathcal{F}$, a stack cS of callee functions with empty specifications, a list aF of functions annotated with generated specifications. It performs symbolic execution throughout the entire procedure, recursively invoking the generation of specifications for callee functions and of loop invariants as needed. Upon termination, it produces an annotated version of $\mathcal{F}$ equipped with its specification, including the pre-/postcondition and loop invariants.

Algorithm 1: Function Specification Generation

Input: $\mathcal{F}$, callStack cS , annoFuncs aF
Output: Function $\mathcal{F}$ with specification $\mathcal{S}$

```

1 if  $\mathcal{F} \in aF$  then
2   return
3  $cS.push(\mathcal{F})$ ;
4  $lS \leftarrow \text{InitLoopStack}()$ ;
5  $P \leftarrow \text{DefaultPre}(\mathcal{F})$ ;
6  $p_{start} \leftarrow p_0$ ;
7  $SEStart(\mathcal{F}, P, p_{start})$ ;
8 while  $p \neq p_{-1}$  do
9    $P \leftarrow SP(\mathcal{F}, P, p_{start}, p)$ ;
10   $p_{start} \leftarrow p$ ;
11  if  $p = p_{loop}$  then
12     $\mathcal{F} \leftarrow \text{LoopInvGen}(\mathcal{F}, p, lS, cS, aF, P, LLM)$ ;
13  if  $p = p_{callee}$  then
14     $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$ ;
15   $SEStart(\mathcal{F}, P, p_{start})$ ;
16  $\mathcal{S} \leftarrow SP(\mathcal{F}, P, p_{start}, p_{-1})$ ;
17  $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{S})$ ;
18  $aF.append(cS.pop())$ ;
19 return  $\mathcal{F}$ ;
```

Algorithm 1 is explained as follows. If $\mathcal{F}$ is already in aF , it returns directly (Lines 1-2). Otherwise, it is added to the call stack cS , which stores functions encountered during execution that lack specifications, and a loop stack lS is initialized (Lines 3-4). They are used to handle nested function calls and loops respectively. Lines 5-6 initialize the default precondition at the entry point p_0 (denoted by P) and sets the starting point for symbolic execution to be p_0 (denoted by p_{start}). Function DefaultPre assigns initial symbolic values to all atomic memory locations reachable from variables in scope at p_0 (typically the function's parameters). Symbolic execution of $\mathcal{F}$ then starts from p_{start} with precondition P by invoking $SEStart$ (Line 7), which will stop whenever a loop or a function call is encountered. If the symbolic execution pauses at p , which is not the exit point of $\mathcal{F}$ (denoted by p_{-1}), the postcondition P at p is obtained via SP based on the symbolic execution results, and p_{start} (the starting point for resuming symbolic execution in the future) is reset to p (Lines 9-10). If it is a loop (Lines 11-12), LoopInvGen is invoked to synthesize a loop invariant, with P as the loop precondition and p as the loop entry, and returns the updated $\mathcal{F}$ that annotates the loop with its invariant. If it is a function call (Lines 13-14), FuncSpec is recursively invoked to obtain the specification of the callee function. The use of call stacks enables a compositional approach to handle nested function calls by first deriving, then leveraging these generated function specifications across different calling contexts. For both of the cases, the symbolic execution will re-proceed from p_{start} with the corresponding annotations (Line 15). When the symbolic execution reaches the function's exit, the algorithm derives the postcondition $\mathcal{S}$ of $\mathcal{F}$, adds the annotated $\mathcal{F}$ with its generated specifications to aF , and finally returns (Lines 16-19).

IV. LOOP INVARIANT GENERATION

The general loop invariant generation problem can be defined as follows:

Definition 1. Given a loop $l: \text{while } B \text{ do } C$ of program $\mathcal{P}$, a precondition P that holds at entry l , and an (optional) postcondition Q , the loop invariant generation problem is to find a logical formula Inv that satisfies the following conditions:

- **Base:** $P \Rightarrow Inv$, i.e., the invariant Inv holds initially;
- **Preservation:** $\{Inv \wedge B\} C \{Inv\}$, i.e. if Inv holds at the start of a loop iteration and loop condition B is true, executing loop body C preserves Inv ;
- **Termination:** $Inv \wedge \neg B \Rightarrow Q$, i.e. upon loop termination, B turns false, Inv remains true and implies postcondition Q .

Among these conditions, the postcondition serves as the verification target, whose objective is to synthesize sufficiently strong invariants that imply it. Our invariant generation algorithm, presented below, is designed to produce strong invariants even in the absence of postconditions. We achieve this through a combination of symbolic execution and Large Language Models (LLMs), using equality-based invariant templates to precisely capture relationships between symbolic

states. The effectiveness of this approach is experimentally demonstrated in Section VI-E0c.

Algorithm 2: Loop Invariant Generation LoopInvGen

Input: $\mathcal{F}$, p_{loop} , loopStack lS , callStack cS , annoFuncs aF , loop precondition P , LLM
Output: Function $\mathcal{F}$ annotated with loop invariant $\mathcal{I}$ for loop $\mathcal{F}_{loop}$
// $\mathcal{F}_{loop}$ is loop body for loop at p_{loop} in $\mathcal{F}$
1 ThinkInNaturalLanguage($\mathcal{F}_{loop}$, LLM);
2 $lS.push(\mathcal{F}_{loop})$;
3 ($\mathcal{F}_{loop}$, P);
4 $\mathcal{F}_{loop}$, uV , $nV \leftarrow \text{LoopAnalysis}(\mathcal{F}_{loop}, P, lS, cS, aF, LLM)$;
5 $lS.pop()$;
6 Assert($lS.empty()$);
7 $\mathcal{T} \leftarrow \text{LoopInvTemGen}(\mathcal{F}_{loop}, P, uV, nV)$;
8 $\mathcal{I} \leftarrow \text{OuterLoopLLMCall}(\mathcal{F}, \mathcal{F}_{loop}, P, \mathcal{T}, LLM)$;
9 $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{I})$;
10 **while** $\neg \text{Verify}(\mathcal{F})$ **do**
11 $\mathcal{F} \leftarrow \text{LoopInvRefinement}(\mathcal{F}, LLM)$;
12 **return** $\mathcal{F}$;

A. Invariant Generation for Loops

Algorithm 2 presents a procedure for the invariant generation of a loop in $\mathcal{F}$. Among the inputs, p_{loop} denotes the entry point of the loop in $\mathcal{F}$, P is the precondition before p_{loop} obtained through previous symbolic execution, the large language model (LLM) serves as the reasoning component. Other parameters remain as previously defined.

First of all (Line 1), the original outermost $\mathcal{F}_{loop}$ is sent to LLM for chain-of-thought in natural language, where LLM were asked few questions for comprehensive understanding. $\mathcal{F}_{loop}$ is pushed into the loop stack and analyzed by LoopAnalysis, which performs the symbolic execution of the loop body to obtain the set of unchanged and non-inductive variables and furthermore recursively handle all inner loops and annotates their invariants into $\mathcal{F}_{loop}$, and after that, the outermost loop $\mathcal{F}_{loop}$ is popped from the stack. Third (Lines 6-8), the invariant for the outermost loop is generated: The results obtained from LoopAnalysis serve as inputs to LoopInvTemGen, which generates a set of invariant templates corresponding to the current loop. Based on these invariant templates, the invariant synthesis for the outermost loop is performed by invoking the LLM-based procedure OuterLoopLLMGen. At the end (Lines 9-11), LoopInvRefinement carries out verification-driven refinement, where the LLM interacts iteratively with the verifier to repair and strengthen the generated invariants until they are verified.

B. Loop Analysis

Algorithm 3 implements LoopAnalysis, which generates invariants for nested loops in $\mathcal{F}_{loop}$ and identifies its unchanged and non-inductive variables. It performs symbolic execution on a single-iteration unfolding $\mathcal{F}_{unfolded}$ (Lines 1-4). The key difference with FuncSpec lies in the handling of nested loops: whereas outer loops use symbolic execution-derived loop preconditions and templates to guide LLM-based

Algorithm 3: loop Analysis LoopAnalysis

Input: $\mathcal{F}$, precondition Pre , loopStack lS , callStack cS , annoFuncs aF , LLM
Output: $\mathcal{F}$, unchanged and noninductive variables uV , nV
1 $\mathcal{F}_{unfolded} \leftarrow \text{UnfoldLoop}(\mathcal{F}_{loop})$;
2 $P \leftarrow Pre$;
3 $p_{start} \leftarrow p_{unfolded_0}$;
4 $\text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
5 **while** $p \neq p_{unfolded_n}$ **do**
6 *... // Same to FuncSpec, reset new precondition and starting point*
7 **if** $p = p_{nestedLoopEntry}$ **then**
8 $lS.push(\mathcal{F}_{nested})$;
9 **if** $p = p_{nestedLoopExit}$ **then**
10 $\mathcal{F}_{nested} \leftarrow lS.pop()$;
11 $\mathcal{F} \leftarrow \text{NestedLoopLLMCall}(\mathcal{F}, \mathcal{F}_{nested}, LLM)$;
12 **if** $p = p_{callee}$ **then**
13 $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$;
14 $\text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
15 *... // Same to FuncSpec, construct the final postcondition $\mathcal{S}$*
16 $uV \leftarrow \text{FetchUnchangeVars}(\mathcal{S})$;
17 $nV \leftarrow \text{FetchNonInductiveVars}(\mathcal{S})$;
18 **return** $\mathcal{F}, uV, nV$;

invariant generation (OuterLoopLLMCall), nested loops are processed purely via LLM (NestedLoopLLMCall), which infers invariants directly from the code context without relying on preconditions and templates, due to the lack of well-defined preconditions of nested loops. As shown in Algorithm 3, nested loops are processed in a bottom-up manner using a loop stack lS , while function calls are handled consistently with FuncSpec with the help of a call stack (Lines 7-13). Upon completion of the symbolic execution of the outermost loop body, the final postcondition is derived, and all nested loops are annotated with the generated invariants; This postcondition is then used to identify unchanged and non-inductive variables within the outermost loop (Lines 15-17).

C. Template Generation

The LoopInvTemGen procedure takes as inputs the loop body, the loop precondition and the sets of unchanged and non-inductive variables, and generates a set of invariant templates. These templates have parameters partially determined by symbolic execution results, while the remaining undetermined parameters are completed using the subsequent LLM synthesis method (i.e. OuterLoopLLMCall in Algorithm 2). Below we introduce five forms of invariant templates.

- **Unchanged variables:** For any variable $v \in uV$ and any execution path π_j to the loop entry p , v retains its symbolic value $\sigma_{p, \pi_j}(v)$ at the loop entry:

$$PC_{\pi_j} \Rightarrow v = \sigma_{p, \pi_j}(v)$$

- **Loop skip conditional property:** For a path π_j to p and a loop condition B , if B for path π_j is false, the loop is skipped without altering variables:

$$\neg(B \wedge (PC_{\pi_j} \wedge \sigma_{p, \pi_j}(v))) \Rightarrow v = \sigma_{p, \pi_j}(v)$$

- **Non-inductive variables:** For a non-inductive variable v (whose value is updated independently of its value from the previous iteration) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \sigma_{p,\pi_j})) \Rightarrow (v = \sigma_{p,\pi_j}(v)) \vee \Phi(v)$$

where $\Phi(v)$ is the inductive invariant to be synthesized.

- **Inductive variables:** For an inductive variable v (which is neither unchanged nor non-inductive) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \sigma_{p,\pi_j})) \Rightarrow \Phi(v)$$

- **Verification goals:** In addition to variable-specific templates, we also introduce templates corresponding to each verification goal. For a verification goal g and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \sigma_{p,\pi_j})) \Rightarrow \Phi(g)$$

This template-driven approach is crucial for our loop invariant generation algorithm. By leveraging precise information derived from symbolic execution, it simplifies the LLM's task, shifting from full invariant generation from scratch to completing invariant templates. It also offers structural guidance, helping the LLM produce formulas that conform to the required ACSL logical syntax. More importantly, this approach ensures the generation of *strong invariants* that capture all relevant variable behaviors across every execution path, thereby closely over-approximating the set of all possible variable updates within the loop.

D. Loop Invariant Refinement

Loop invariants generated by the LLM are not always valid (i.e., able to pass formal verification) or sufficiently strong to guarantee the verification goal. Additionally, while our template-based approach improves the precision of invariant generation, it also constrains the diversity of candidate invariants. To alleviate these issues, we propose a fine-grained iterative refinement framework, as shown in Algorithm 4, that combines verifier feedback with the LLM's self-correction ability. The core insight of our approach is that different types of verification failures require fundamentally different repair strategies. Rather than treating all errors uniformly, our framework implements a four-layer hierarchical error-handling mechanism that applies increasingly aggressive refinement strategies based on error severity and complexity.

During the refinement process, depending on the verifier's feedback, the framework applies different strategies organized into four layers: **Layer 1 - Syntax Repair** (Lines 11-12): Syntax errors are repaired directly through a dedicated repair call that provides the LLM with precise error messages. **Layer 2 - Adaptive Refinement** (Lines 15-24): For single-category verification failures, targeted modifications are applied: weak invariants (passing Base and Preservation but failing to establish the verification goal) are strengthened; invariants failing Base are weakened; and invariants failing Preservation are adjusted to maintain the inductive structure. **Layer 3 - Regeneration** (Lines 25-26): Invariants failing multiple checks simultaneously are discarded and regenerated from scratch with fresh guidance. **Layer 4 - Elimination**

(Lines 4-10): If repeated refinement attempts fail to converge, a final elimination step removes consistently invalid invariants, retaining the largest valid subset. At each step, the LLM is guided by a specialized prompt that encodes these layer-specific refinement strategies along with the precise error context. This hierarchical approach effectively steers the model away from incorrect specifications while maximizing the probability of convergence toward valid solutions.

Algorithm	4:	Iterative	Refinement
LoopInvRefinement			
Input: $\mathcal{F}$ with candidate loop invariants $\mathcal{I}_0$, <i>LLM</i>			
Output: $\mathcal{F}^*$ with verified invariants $\mathcal{I}^*$			
1	$\mathcal{I} \leftarrow \mathcal{I}_0$;		
2	$\mathcal{F}^- \leftarrow \mathcal{F} - \mathcal{I}_0$		
	// Run verifier and collect errors		
3	while $Errors \leftarrow \neg \text{Verify}(\mathcal{F})$ do		
4	if reach iteration limit then		
	// Find largest valid subset in $\mathcal{I}$		
5	while $\mathcal{I}$ remains invalid do		
6	foreach $e \in Errors$ do		
7	$i \leftarrow$ associate loop invariant with e ;		
8	$\mathcal{I} \leftarrow \text{Elimination}(\mathcal{I}, i)$;		
9	$\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{I})$;		
10	break ;		
11	if $Errors = \text{syntax error}$ then		
12	$\mathcal{I} \leftarrow \text{RepairCallLLM}(\mathcal{I})$;		
13	else		
14	$guidance \leftarrow \{\}$;		
15	if $Errors = \text{verification goals fail}$ then		
	// Only termination failed		
16	$guidance \leftarrow guidance \cup \text{strengthGuide}(\mathcal{I})$;		
17	else		
18	foreach $e \in Errors$ do		
19	if $e = \text{loop invariant error}$ then		
20	$i \leftarrow$ associate loop invariant with e ;		
21	if e not base then		
22	$guidance \leftarrow$		
	$guidance \cup \text{WeakenGuide}(i)$;		
23	if e not preservation then		
24	$guidance \leftarrow$		
	$guidance \cup \text{AdjustGuide}(i)$;		
25	else		
26	$guidance \leftarrow$		
	$guidance \cup \text{RegenGuide}(i)$;		
27	$\mathcal{I} \leftarrow \text{RefineCallLLM}(\mathcal{I}, guidance)$;		
28	$\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{I})$;		
29	return $\mathcal{F}$		

V. IMPLEMENTATION

A. Toolchain Integration and Data Flow

The implementation architecture and the underlying data flow between the components are illustrated in Fig. 3. The process initiates with the C source code entering the **Executor**, which performs symbolic execution to generate *SE Info* in the form of symbolic states (pairs of path conditions and symbolic stores). This symbolic information is then passed to the **Translator**, which transforms the symbolic states in the syntax of the **Executor** into Natural Language descriptions

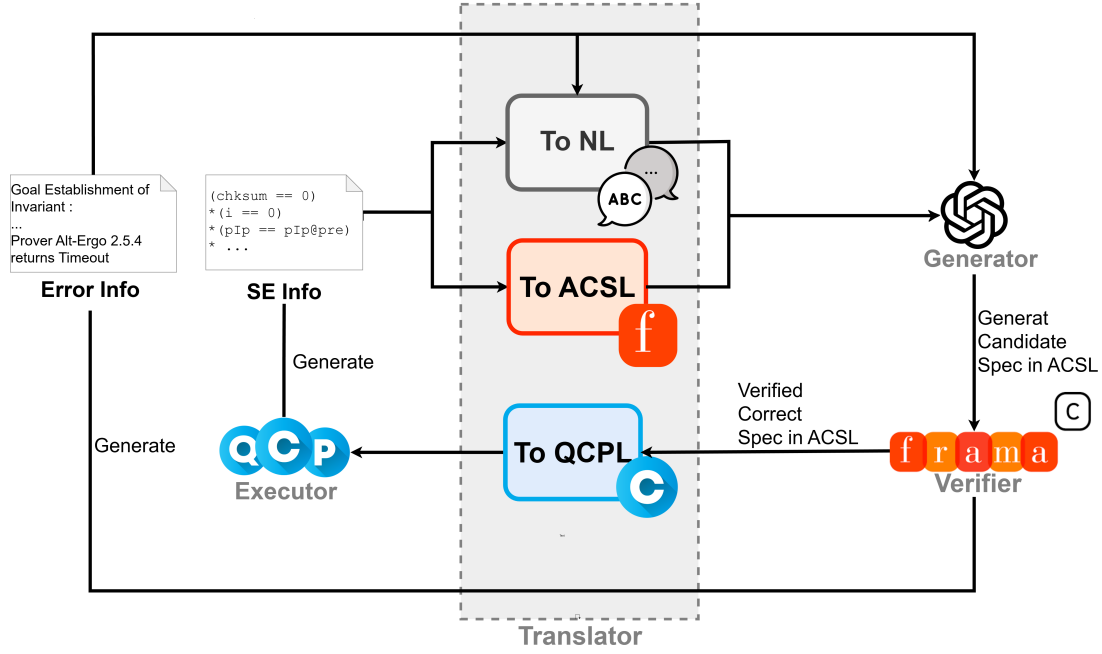


Fig. 3. The Implementation Design Framework

and the input syntax of the formal **Verifier**. Based on these translated representations, the **Generator** synthesizes candidate specifications, which are subsequently sent to the **Verifier** for automated verification. If a specification is successfully verified, it is translated back and fed into the **Executor** for further symbolic execution, allowing the system to progressively cover the entire program. If verification fails, the resulting *Error Info* is translated into Natural Language together with the original error context and fed back to the **Generator** for iterative refinement. This feedback-driven loop tightly couples symbolic execution, specification synthesis, and formal verification, ensuring that the final output specification is both semantically meaningful and formally sound.

This architecture is instantiated using QCP [17], [18] as the symbolic execution engine and Frama-C [19] as the formal verifier, a combination motivated by their complementary strengths. QCP serves as our foundational symbolic executor because it generates Coq-verified postconditions and exposes symbolic execution results at arbitrary control points in a logical form. Moreover, QCP provides strong support for programs manipulating complex data structures, along with a rich library of verified inductive predicates and lemmas, which aligns well with our goal of generating formally precise specifications. In contrast, widely used symbolic execution tools such as Klee [20] neither expose intermediate symbolic states at arbitrary program locations nor represent execution states in a logic-oriented form suitable for direct specification synthesis. However, QCP requires manual annotations for loop invariants and callee function pre/postconditions, and may introduce unproven lemmas that require additional Coq proofs. To mitigate these limitations, we leverage the **Generator** to automatically synthesize candidate invariants, and integrate Frama-C as the **Verifier** to discharge proof obligations automatically. Frama-C’s ACSL assertion language provides an

LLM-friendly interface for specification generation, while its integration with SMT solvers enables scalable and automated verification. Together, QCP and Frama-C form a synergistic execution–verification backbone that supports our iterative, LLM-in-the-loop specification generation workflow.

The **Translator** serves as a critical interoperability layer in our system, mediating the interaction between the **Executor** (QCP), the **Generator**, and the **Verifier** (Frama-C). It establishes a bidirectional translation between ACSL and the QCPL (QCP’s annotation language), enabling symbolic execution and formal verification to be tightly integrated in an iterative loop. During the LoopInvGen phase, specifications in ACSL are translated into QCPL so that QCP can interpret them consistently across loop and function boundaries. Conversely, symbolic execution results produced by QCP are translated back into ACSL and discharged by Frama-C through automated verification. In this way, the Translator enables continuous information flow between execution and verification. A key design choice of the **Translator** is that the internal QCPL representation is never exposed to the LLM. Instead, symbolic execution results are transformed into a combination of Natural Language explanations and ACSL skeletons before being passed to the **Generator**. This transformation shields the model from low-level syntactic and semantic details specific to QCPL, reducing ambiguity and preventing the model from confusing QCPL with ACSL during generation. At the same time, the use of ACSL skeletons provides a pre-structured formal framework, allowing the LLM to focus on logical correctness rather than grammatical well-formedness.

B. Prompt Designs

A distinguishing feature of our approach is the use of **dynamic, differential prompts** that adapt to both the structure

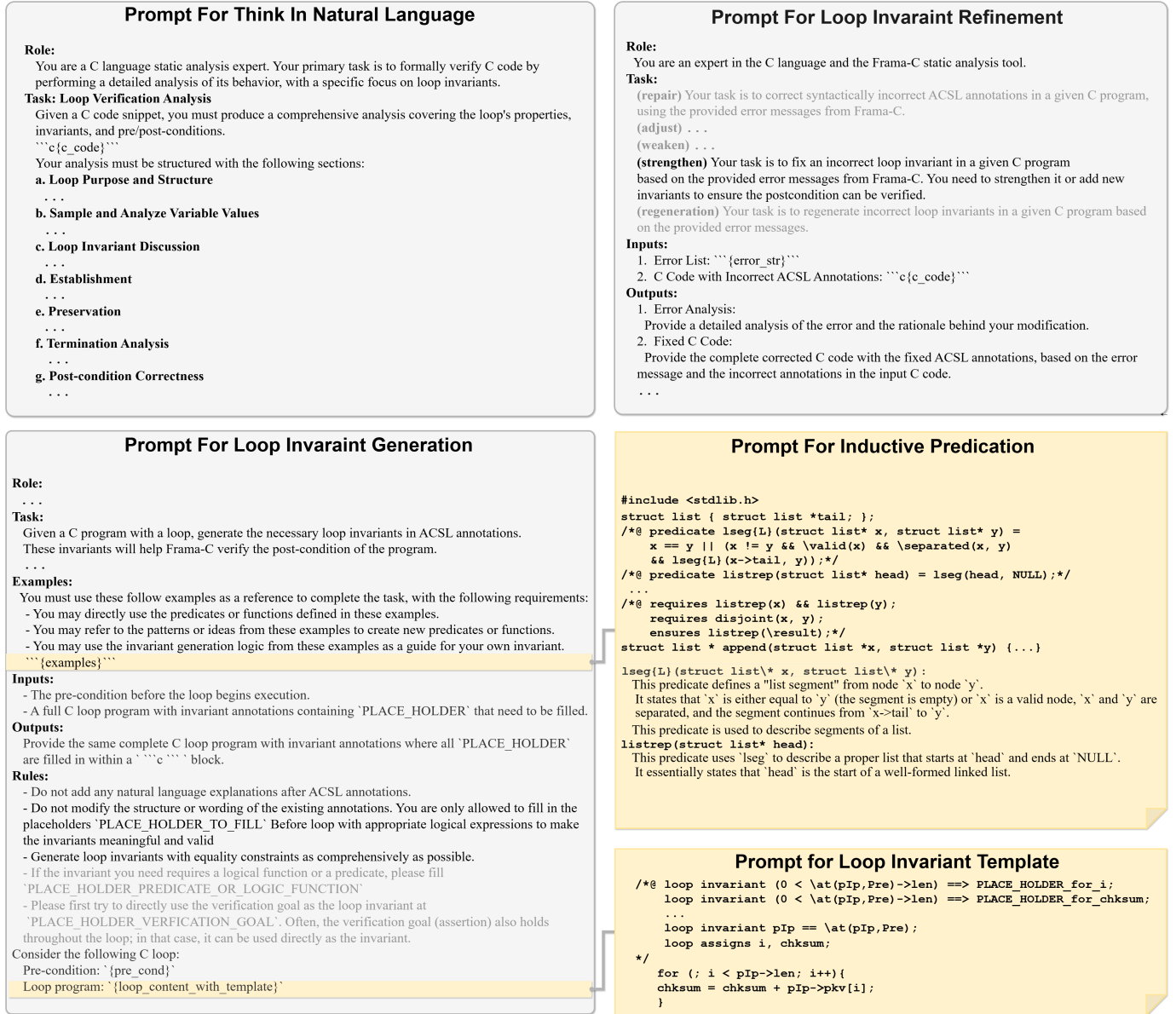


Fig. 4. The Prompt Design

of the program under analysis and the evolving state of the verification process. Rather than relying on a fixed prompt template, our system constructs prompts through two complementary mechanisms: (1) symbolic execution-driven template instantiation, which populates loop invariant skeletons with program-specific variables and path conditions; and (2) differential content routing, which conditionally includes or excludes prompt components based on program characteristics and verification feedback. This adaptive prompt architecture minimizes irrelevant context, focuses the LLM's reasoning on pertinent concerns, and enables more accurate and efficient invariant synthesis. We detail the prompt design for loop invariant generation and inductive predicate synthesis below.

a) Prompt Design for Loop Invariant Generation:

Our prompt design strategy illustrated in Fig.4 is intricately mapped to the multi-stage control flow of Algorithm 2 (LoopInvGen), ensuring that the LLM's role dy-

namically adapts to each specific execution phase. The process initiates with the "Think In Natural Language" prompt (ThinkInNaturalLanguage, Line 1) to establish initial program comprehension, followed by the "Loop Invariant Generation" prompt (OuterLoopLLMCall, Line 8), which constrains the LLM to fill precise, symbolic execution-derived Loop Invariant Template, providing a structured skeletal format for ACSL annotations (LoopInvTemGen, Line 7). Crucially, the differential prompting logic is applied across both stages: during the generation phase, prompts and placeholders for arrays or inductive predicates are omitted when irrelevant to the program context; during the refinement procedure (LoopInvRefinement, Line 10), the system dynamically selects appropriate instruction sets—such as repair, weakening, adjustment, strengthening, or regeneration—ensuring the LLM receives only the guidance pertinent to the specific logical deficiency identified by the verifier, rather than the full space

of possible corrections.

b) Prompt Design for Inductive predication: For numerical programs, the specifications can be expressed using arithmetic formulas in first-order logic, and our symbolic execution-guided LLM approach provides high accuracy for inferring loop invariants. However, programs involving recursive data structures or inductive operations require specifications expressed using inductive predicates or recursive functions. In these cases, LLMs often struggle not with understanding program semantics, but with correctly adhering to the syntactic and semantic conventions of inductive specifications in the target specification language, frequently producing assertions with syntax errors or uninterpreted functions that lack formal definitions.

To address this issue, we conditionally include auxiliary guidance in the prompt (see Fig. 4) only when the program manipulates relevant data structures. Specifically, we provide two textbook-style illustrative examples in Frama-C format, drawn from publicly available and well-established sources: one over arrays and one over single linked lists, taken from the QCP and Frama-C repositories, respectively. These examples are not used to convey program-specific properties, nor to bias the model toward particular benchmark structures. Instead, they serve solely to calibrate the LLM to the proper syntactic forms and intended semantic usage of inductive predicates and functions, thereby reducing hallucinations and malformed specifications.

With this lightweight calibration, the LLM is able to correctly instantiate and synthesize inductive predicates when required, while relying on symbolic execution to infer program-specific behavior. As a result, our approach can handle programs manipulating single linked lists and arithmetic inductive computations (e.g., factorial), which remains beyond the capabilities of [13]—all without resorting to training or large-scale external data.

VI. EXPERIMENTS

This section evaluates our approach for automatic specification generation based on symbolic execution (hereafter referred to as SESpec).

A. Experimental Setup

Benchmark: We evaluate our approach on multiple prominent and widely-adopted benchmarks, which are commonly used by recent state-of-the-art tools. These include: **SyGuS** [21], collected by CODE2INV [10]; **OOPSLA** [22]; **NLA**, collected by LIPuS [11]; the datasets from **SV-COMP** and **Frama-C**, collected by AutoSpec [13]. In addition, we constructed three new benchmarks to capture more program scenarios: The **plp** set is extracted from an actual Sun Search System used in aerospace control software, involving pointer and struct manipulations as well as complex branching structures. We carefully adapt and reformulate **plp** programs into a format acceptable to Frama-C and QCP, thereby establishing a new dataset, which constitutes one of our contributions; the **list-S** set is derived from the LIG-MM [23] benchmark and includes all singly linked list programs from SLING and

SV-COMP; and the **numer-S** set is a curated collection of additional programs, comprising 20 linear and 20 nonlinear loops primarily from CLAUSE2INV [20], which exceed the difficulty of those in **SyGuS** and **NLA**.

Verifier: All experiments are performed using Frama-C version 29.0 (Copper). We employ the WP plugin as the verification backend, which enforces a 3-second timeout per query, uses Z3 as the primary SMT solver, and applies the Typed memory model.

Model: Our LLM selection for evaluating specification generation balanced model maturity, prevalence, and cutting-edge performance: **GPT-3.5-turbo** is an earlier, foundational model that serves as an essential baseline; **GPT-4o-mini**, a smaller, established model optimized for efficiency; **GPT-4o**, a widely adopted model for general code comprehension and generation and **Claude 3.7 sonnet**, a stronger hybrid model with advanced reasoning and code understanding. For all experiments, we use a fixed sampling temperature of 0.7 to balance output diversity and stability.

Below we evaluate our approach through four research questions:

- RQ1: What Types of Programs can SESpec Manage?
- RQ2: How does SESpec Perform on Program Specification Generation?
- RQ3: Is SESpec Effective?
- RQ4: What are the Advantages of SESpec Over Other Work?

B. RQ1: What Types of Programs can SESpec Manage?

Table I summarizes the 395 benchmark programs used in our evaluation. They encompass a diverse range of program structures, including linear/non-linear/nested/multi-loops, function calls, along with data structures involving pointer manipulations, and recursively-defined data types such as linked lists. Importantly, our benchmark suite includes both loop-only programs (indicated by **Blue** cells), requiring invariant generation, and programs requiring full specification generation, i.e. loop invariant/pre/postcondition (indicated by **Red** cells).

Overall, SESpec successfully verified 354 out of 395 programs (nearly 90%). The results demonstrate that our method is applicable to a wide range of program types, especially generalizes to real-world industrial cases in the **plp** dataset selected from an actual aero system. Our current approach does not yet cover recursive functions or programs involving library functions and system calls, which we leave as future work.

Furthermore, our evaluation reveals that lines of code (LOC) do not reliably reflect verification difficulty on these benchmarks; instead, the primary challenge arises from the complexity of loop invariants. As shown in our results, SESpec achieves a 97% success rate on Linear Loop and Iteration-Free programs. However, the success rate drops to 77% for Non-Linear Loop programs and to 63% for Nested & Multi-Loop programs. This decline is primarily due to the increased difficulty of synthesizing invariants that capture complex numerical relationships, intricate data structure properties, and interactions among nested or multi-level loop structures.

TABLE I
DATASET OVERVIEW OF PROGRAM TYPES

Program Type	SyGus	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S	#Program	#Solved
Linear Loop	133	30	14	0	20	7	0	0	204	197
Non Linear Loop	0	0	0	30	20	3	0	0	53	41
Nested & Multi Loop	0	16	7	0	0	1	0	0	24	15
Memory & Loop	0	0	2	0	0	17	12	24	53	45
Iteration Free	0	0	0	0	0	20	38	0	58	56
External & Recursive Call	0	0	0	0	0	3	0	0	3	0
#Program	133	46	21	30	40	51	50	24	395	
#Solved	133	40	16	25	34	39	44	23		354
Avg. LoC	19	24	22	21	19	18	57	24		

C. RQ2: How does SESpec Perform on Program Specification Generation?

We evaluate correctness through three metrics:

- **Syntax rate (Syn.):** the percentage of generated specifications that are syntactically correct;
- **Validity rate (Val.):** the percentage of generated specifications that are syntactically correct and valid, meaning that they can pass the formal verification of verifiers such as Frama-C. For example, the generated invariants for loops are valid if they satisfy both the *Base* and *Preservation* conditions;
- **Accuracy rate (Acc.):** the percentage of generated specifications that are syntactically correct, valid, and also satisfy the verification goal. For example, the generated invariants for loops are accurate if they satisfy the *Base*, *Preservation*, and *Termination* conditions.

These metrics are assessed under different Pass@k settings, reflecting the system’s ability to attempt multiple generations until the specifications pass all three metrics.

Table II presents a comprehensive overview of our experimental results.

The observed gaps among different evaluation metrics reveal important characteristics of both benchmark difficulty and model capability. As benchmark complexity increases, the discrepancies between Syn., Val., and Acc. become more pronounced. A similar trend is observed across models: weaker models tend to exhibit larger gaps between these metrics. Notably, Val. does not trivially coincide with Syn. in our results, indicating that the generated specifications are not simplistic invariants (e.g., `true`) or trivial properties that programs would satisfy by default. Instead, our approach aims to capture complete program semantics and functional correctness, which naturally induces a meaningful distinction between syntactic correctness and semantic validity.

Importantly, our method demonstrates consistent effectiveness across different models and significantly narrows the performance gap between stronger and weaker ones. For example, on benchmarks such as **SV-COMP**, **NLA**, and **numer-S**, GPT-4o and Claude achieve identical performance at higher Pass@k levels (Pass@3 and Pass@5), highlighting the capability of our approach to amplify the effectiveness of comparatively weaker

models. Furthermore, the method exhibits rapid convergence under Pass@k evaluation. Performance improvements from Pass@3 to Pass@5 are marginal across several benchmarks, including **NLA**, **numer-S**, and **pIp**, suggesting high resource efficiency by reducing the need for excessive evaluation cycles.

The results also indicate comparatively lower accuracy on the **OOPSLA**, **SV-COMP**, and **Frama-C** benchmarks. For **OOPSLA** and **SV-COMP**, this is primarily due to the presence of nested and multiple loops, where symbolic execution often fails to derive sufficiently strong preconditions. In addition, the absence of explicit loop conditions in **OOPSLA** further complicates invariant inference. In the case of **Frama-C**, the difficulty mainly stems from complex loop structures involving array manipulations, external function calls, and recursive behaviors, which remain challenging for symbolic execution. Despite these obstacles, our method still achieves notable performance on these challenging benchmarks. Moreover, the consistent improvements from Pass@1 to Pass@5 further underscore the robustness and scalability of our approach in complex verification scenarios.

D. RQ3: Is SESpec Effective?

To assess the individual contributions of each component in our method, we conducted a detailed ablation study using 100 randomly chosen programs, consisting of 70 loop-related and 30 function-related cases. To ensure a fair comparison under controlled computational budgets, we designed the experiment to maintain consistent LLM token usage across all configurations. Specifically, we capped the refinement process at a maximum of 3 iterations.

We compared four distinct configurations: The **Only LLM (Pass@3)** baseline uses the LLM for trivial invariant generation without any external guidance. The **LLM + SE (Pass@3)** variant enhances the LLM with symbolic execution, providing structural templates and program context. **LLM + SE + REFINE (Pass@1)** incorporates our refinement module for iterative improvement. These configurations are designed to facilitate comparison with the incremental approach employed in our work.

The ablation results indicate that using the LLM alone yields high syntactic correctness, but exhibits notably low validity

TABLE II
SPECIFICATION GENERATION PERFORMANCE OVERVIEW

Benchmark	Model	Pass@1 (%)			Pass@3 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGus	GPT-4o-mini	90.98	87.97	78.20	97.74	95.49	88.72	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	97.74	97.74	92.48	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	100.00	99.25	99.25	100.00	100.00	100.00
	Claude 3.7 sonnet	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
OOPSLA	GPT-4o-mini	89.37	78.95	65.41	89.13	86.96	43.48	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	82.61	47.83	86.96	86.96	52.17	86.96	86.96	52.17
	GPT-4o	91.30	69.57	60.87	95.65	76.09	73.91	100.00	82.61	76.08
	Claude 3.7 sonnet	93.48	69.67	63.04	97.83	80.43	76.09	97.83	93.57	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	80.95	80.95	42.86	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	71.43	100.00	100.00	76.19
	Claude 3.7 sonnet	100.00	100.00	71.43	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67	96.67	96.67	86.67
	Claude 3.7 sonnet	96.67	96.67	86.67	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	90.00	65.00	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 sonnet	100.00	97.50	85.00	100.00	100.00	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	62.75	50.98	49.02	64.71	50.98	49.02
	GPT-3.5-turbo	77.08	60.42	52.08	83.33	62.50	54.17	83.33	62.50	54.17
	GPT-4o	82.35	72.55	66.67	86.27	76.47	70.59	86.27	78.43	72.55
	Claude 3.7 sonnet	86.27	78.43	72.55	92.16	82.35	76.47	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	87.50	65.00	92.50	92.50	72.50	92.50	92.50	75.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	72.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	76.00	94.00	78.00	78.00
	Claude 3.7 sonnet	100.00	88.00	82.00	100.00	90.00	88.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	75.00	45.83	41.67	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	87.50	83.33	75.00	91.67	87.50	79.17
	Claude 3.7 sonnet	95.83	83.33	75.00	100.00	100.00	95.83	100.00	100.00	95.83

and accuracy. Across both experimental settings, the integration of symbolic execution (SE) leads to substantial improvements in validity and accuracy—with an average increase of 30%—while only slightly reducing syntactic correctness in the first setting (less than 2%). The incorporation of the refinement step (REFINE) further elevates performance, achieving perfect scores across all metrics in the first setting and delivering an average improvement of 20% in validity and accuracy in the second.

Overall, the ablation study confirms that each component contributes positively to the final outcome, with symbolic execution and refinement yielding the most significant gains.

E. RQ4: What are the Advantages of SESpec Over Other Work?

a) Overall Comparison With LLM Approaches: Table III compares three representative LLM-based approaches:

AutoSpec, ACInv, and our tool SESpec. First, on the four datasets originally introduced by AutoSpec, our method consistently outperforms ACInv across both GPT-3.5-turbo and GPT-4o. Moreover, the performance gains from upgrading the underlying model are similar for ACInv and SESpec, highlighting that our workflow is comparably scalable yet significantly more effective.

In contrast, AutoSpec exhibits negligible improvement when transitioning from GPT-3.5-turbo to GPT-4o. This stagnation stems from AutoSpec’s reliance on few-shot prompting combined with exhaustive sampling and verification-based filtering. By prioritizing the generation of massive candidate pools rather than leveraging the full reasoning potential of LLMs, AutoSpec hits a performance ceiling that stronger models cannot easily break. Consequently, our reproduction of AutoSpec-4o remains at a similar level to the reported AutoSpec-3.5 results.

Method	Syn. (%)	Val. (%)	Acc. (%)
Only LLM	97.50	37.50	28.75
LLM + SE	95.71	62.86	60.00
LLM + SE + REFINE	95.71	95.71	87.21
Only LLM	66.67	26.67	16.67
LLM + SE	76.67	50.00	46.67
LLM + SE + REFINE	93.33	86.67	86.67

Fig. 5. Ablation Study. Blue cells: loop invariant generation. Red cells: program specification generation.

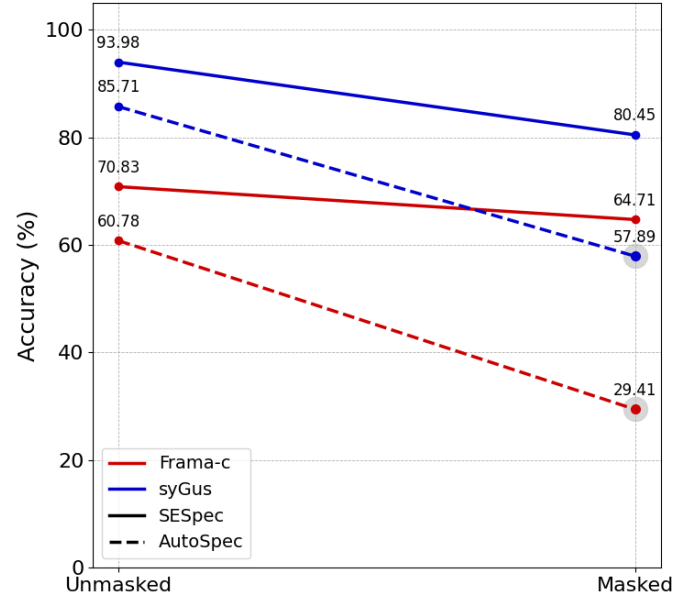


Fig. 6. The impact of masking verification goals. Gray-shaded dots are reproduced results.

TABLE III
SPECIFICATION GENERATION PERFORMANCE COMPARISON. THE GRAY-SHADED CELLS REPRESENT RESULTS REPRODUCED BY US.

Method	SyGus	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S
CODE2INV	54.9%	—	—	—	—	—	—	—
HOLA	—	93.5%	—	—	—	—	—	—
CLN2INV	93.2%	—	—	—	—	—	—	—
LiPus	93.2%	—	—	83.3%	—	—	—	—
CLAUSE2INV	99.2%	—	—	93.3%	—	—	—	—
AutoSpec-3.5	85.7%	82.6%	76.2%	70.0%	75.0%	60.8%	14.0%	0.0%
AutoSpec-4o	86.5%	71.7%	66.7%	70.0%	72.5%	68.6%	22.0%	0.0%
AutoSpec-Claude-3.7	88.7%	80.4%	66.7%	60.0%	55.0%	74.5%	4.0%	0.0%
ACInv-3.5	71.4%	45.7%	42.9%	—	—	34.5%	—	—
ACInv-4o	79.9%	76.1%	76.2%	—	—	48.3%	—	—
SESpec-3.5	93.2%	52.2%	66.7%	56.7%	67.5%	51.0%	76.0%	16.7%
SESpec-4o	100.0%	76.1%	76.2%	86.7%	82.5%	72.6%	78.0%	79.2%
SESpec-Claude-3.7	100.0%	87.0%	76.2%	86.7%	85.0%	76.5%	88.0%	95.83%

Conversely, SESpec demonstrates steady and reliable gains as the underlying model improves. Our workflow demands complex reasoning capabilities—specifically, simulating execution via a “Thinking In Natural Language” procedure and iteratively repairing, strengthening, or weakening invariants based on verification feedback. This complexity exceeds the instruction-following limits of GPT-3.5-turbo, resulting in less competitive performance on the weaker model. The use of symbolic execution reduces this complexity by providing structured symbolic constraints, enabling superior models like GPT-4o to achieve significant performance gains through

deeper reasoning, eventually surpassing AutoSpec across almost all benchmarks.

This distinction is further illustrated by the performance on complex datasets like **pIp**. We observe that AutoSpec-Claude-3.7 performs poorly despite the model’s advanced capabilities. Although Claude possesses superior mathematical logic and coding abilities, it often struggles to generate correct ACSL syntax, leading to high failure rates. SESpec mitigates this issue by utilizing an ACSL skeleton to represent symbolic states, which structurally constrains the output to avoid syntactic errors while fully leveraging the model’s logical reasoning

strengths.

Finally, we emphasize the difference in experimental setup. SESpec operates primarily in a zero-shot setting. In contrast, AutoSpec relies on handcrafted few-shot examples as an indispensable part of its workflow. This highlights the stronger generalization ability of SESpec: it achieves competitive or superior performance through autonomous reasoning rather than depending on few-shot engineering.

b) Detailed Results over Benchmarks on SESpec: Our method demonstrates consistent advantages over SMT-based approaches (combined with machine learning) across several major numerical invariant benchmarks. On the **SyGuS** dataset introduced by CODE2INV, our approach achieves a 100% success rate, outperforming the state-of-the-art SMT-based tool CLAUSE2INV. For the nonlinear numerical **NLA** benchmark proposed by LiPus, our method matches the success rate of LiPus and remains competitive, although it trails CLAUSE2INV by 10%. Furthermore, on the **OOPSLA** dataset, proposed by the abductive inference-based invariant generator HOLA and designed for numerical loops without explicit conditions, including nested and multiple loops, our method achieves a success rate of 87% on this challenging case. However, these existing approaches based on SMT solvers are specialized for numerical loops.

By integrating symbolic execution, our method demonstrates significant advantages over purely neural baselines like the state-of-the-art AutoSpec and ACInv on numerical benchmarks. It achieves a 15% higher accuracy on **SyGuS** and noticeably outperforms them on **NLA** and **numer-S**. On the **OOPSLA** benchmark, our approach achieves a success rate comparable to ACInv and is only marginally lower than AutoSpec (With the Claude-3.7 model, we achieve 5% higher rate). All three methods perform equally on **SV-COMP** benchmark. We speculate that this is due to the dataset's non-uniform distribution of problem difficulty, with challenging nested and multi-loops that remain unsolved by all existing approaches. The above results demonstrate how symbolic guidance enables more precise specification generation that pure neural methods cannot achieve.

On the **Frama-C** benchmark for functional program verification, our method noticeably outperforms AutoSpec by incorporating support for inductive predicates, enabling more expressive specification inference. When evaluated on the **plp** benchmark from the real-world Sun Search system, which features extensive use of structures and pointer operations, our approach achieves an average 50% higher success rate compared to AutoSpec, thanks to the combination of a flattened memory model and precise symbolic execution. Furthermore, our method attains high performance on the **list-S** benchmark comprising linked lists, a capability absent in AutoSpec due to its lack of support for inductive data types. These results demonstrate that our approach offers robust and generalizable support for both closed struct types (via flattened unfolding) and inductive data structures (through inductive predicates).

c) Specification Generation without Verification Goals: To demonstrate the capability of our tool SESpec on specification generation in the absence of verification goals, we conducted experiments on **SyGuS** and **Frama-C** by deliberately

masking the goals and compared with AutoSpec. We performed GPT-4o with Pass@1 experiments on SESpec and chose their own configuration on AutoSpec [13], respectively, to compare the accuracy of the two and to analyze the impact of masking verification goals. As shown in Figure 6, SESpec maintains relatively stable accuracy across the two settings, decreasing from 93.98% to 80.45%. Similarly, in the **Frama-C** benchmark, the accuracy only drops slightly from 70.83% to 64.71%, further demonstrating the robustness of SESpec without verification goal. By contrast, AutoSpec experiences a drastic decline in accuracy: On the **SyGuS** benchmark, accuracy falls from 85.71% to 57.89%, while on **Frama-C** it decreases sharply from 60.78% to just 29.41%.

The performance drop of SESpec is primarily due to the refinement process, which performs targeted optimization with respect to the specific verification goal. A key finding is that if the improvement from refinement is disregarded in our approach, the performance under the two settings is still nearly identical. This demonstrates that our symbolic execution-guided reasoning ensures high-quality specification generation even in the absence of specific verification targets.

VII. CASE STUDIES

In this section, we present three case studies that demonstrate how SESpec integrates symbolic execution with LLMs to generate formally verified specifications across a range of challenging program patterns. Each case highlights a distinct class of difficulty: (1) a numerical loop exhibiting nondeterministic and non-inductive control flow, which complicates the derivation of semantically faithful invariants; (2) a loop with input-dependent entry behavior that performs inductive operations, requiring recursively defined logic predicates to capture its functional semantics; and (3) a branch-intensive function that manipulates structs and pointers, featuring complex path branches and aliasing constraints. Together, these studies illustrate the generality, precision, and robustness of our approach in handling diverse verification tasks.

For all the case studies, SESpec generates the full specifications, enclosed by `/* @... */`.

A. Case 1: A Numerical Loop from SyGuS

This example in Fig. 7, drawn from the SyGuS benchmark suite, features a numerical loop with nondeterministic control flow. The function initializes a counter variable `c` to zero and repeatedly updates it inside a `while (unknown())` loop under the precondition `n > 0`.

The verification goal $(c = n) \Rightarrow (c \geq 0)$ is deliberately weak: it is implied by the basic non-negativity of `c` and thus reveals little about the loop's functional behavior. AutoSpec can discharge this assertion, but the resulting specifications are largely goal-driven and semantically uninformative. Concretely, AutoSpec produces vacuous or overly coarse invariants. For example, the existential implication $\exists k. (0 \leq k \leq n \Rightarrow c = k)$ can be satisfied by choosing an out-of-range `k` and therefore imposes no real constraint on `c`, while $\exists k. (0 \leq k \leq n \wedge c = k)$ essentially reduces to the interval bound $0 \leq c \leq n$. Such invariants are sufficient for the given


```

int unknown();

/*@
  logic integer product(integer begin, integer end) =
    (end < begin) ? 1 : product(begin, end - 1) * end;
*/

/*@
  requires \true;
  assigns \nothing;
  ensures (n < 1) ==> \result == 1;
  ensures (n >= 1) ==> \result == product(1, n);
*/
int factorial9(int n) {
  int i = 1;
  int f = 1;

  /*@
    loop invariant (1 <= \at(n,Pre)) ==> (1 <= i <= n + 1);
    loop invariant (1 <= \at(n,Pre)) ==> (f == product(1, i - 1));
    loop invariant (! (1 <= \at(n,Pre))) ==> ((f == 1) && (i == 1) && (n == \at(n,Pre)));
    loop invariant n == \at(n,Pre);
    loop assigns f, i;
  */
  while (i <= n) {
    f = f * i;
    i = i + 1;
  }

  return f;
}

void goo9() {
  int t = factorial9(5);
  //@ assert t == 120;
}

```

Fig. 8. An inductive program from Frama-C

Purely LLM-based approaches consistently struggle here. They fail to generate correct memory preconditions, especially the separation constraint `\separated(pIp, pIp->ret)` required to avoid aliasing issues. And, even when provided with correct preconditions, their postconditions typically cover only a fraction of the 32 paths, leading to severely over-approximating specifications.

SESpec overcomes these limitations through systematic symbolic execution. All 32 paths are explored, and each postcondition is guarded by precise path conditions (e.g., `-pIp->abs[2] <= pIp->abs[1] && pIp->abs[0] <= pIp->abs[1] && ...`), ensuring complete coverage. The preconditions including memory validity and separation constraints are also automatically derived.

In cases when path explosion becomes prohibitive by the symbolic execution engine, SESpec falls back to LLM-generated over-approximations that are formally verified. This example demonstrates that SESpec can handle real-world programs with nontrivial branching structures that defeat both pure-LLM and prior approaches.

VIII. THREATS TO VALIDITY

a) Data Leakage: A major threat to the validity of specification generation approaches lies in unintended bias, data leakage, and overfitting to evaluation benchmarks, especially in the context of program verification where truly domain-specific data are scarce. Training-based or retrieval-augmented methods are particularly vulnerable to these issues, as benchmark adjacent corpora, curated examples, or synthetically generated programs may implicitly encode benchmark specific patterns, leading to over-optimization for a given dataset rather than genuine semantic understanding.

We mitigate these threats by deliberately avoiding all forms of external supervision and dataset dependency. Our approach does not involve model fine-tuning, retrieval from large-scale external corpora, synthesized datasets, or few-shot prompting. Consequently, the model cannot adapt its behavior to any specific benchmark distribution or overfit to particular datasets. Instead, we exclusively rely on the in-context reasoning capabilities of pretrained large language models, guided by symbolic execution results derived solely from the target program. These symbolic states provide precise semantic context without introducing any external information about benchmark structure or expected specifications.

As a result, the generated specifications are grounded purely in the semantics of the analyzed program rather than in prior exposure to benchmark-related data. This design substantially strengthens the internal validity of our evaluation by minimizing bias, preventing data leakage, and avoiding overfitting to any dataset, ensuring that observed performance gains stem from the proposed symbolic execution-guided reasoning process rather than dataset-specific artifacts.

b) Benchmark and Baseline Selection: To ensure a fair and comprehensive evaluation, we selected benchmarks that align with the current state-of-the-art in program verification. Our evaluation includes benchmarks used in the works we compare against, specifically the AutoSpec benchmark and two external datasets from Lipus and Clause2Inv. These benchmarks are representative of the field's development, featuring comparable levels of loop complexity and lines of code (LoC). To further stress-test our approach, we introduced an original benchmark **pIp** that feature significantly more complex data structures and intricate control-flow patterns than existing sets.

Regarding the exclusion of SpecGen [15] from our direct comparison, we note two primary reasons: first, a fundamental

```

typedef struct __TripleAbsMax
{
    int abs[3];
    int tmax;
    int* ret;
} TripleAbsMax;

/*@
requires \valid(pIp);
requires \valid(pIp->abs+(0..2)) ;
requires \valid(pIp->ret);
requires \separated(pIp, pIp->ret);

ensures pIp->ret == \old(pIp->ret);
ensures -pIp->abs[2] <= pIp->abs[1] && pIp->abs[0] <= pIp->abs[1] && pIp->abs[2] < 0 && pIp->abs[1] >= 0
&& pIp->abs[0] >= 0 ==> pIp->tmax == pIp->abs[1] && *\old(pIp->ret) == pIp->abs[1];
... all 32 paths

assigns pIp->tmax, *(pIp->ret);
*/
void TripleAbsMaxFun(TripleAbsMax *pIp)
{
    int absfx1 = pIp->abs[0];
    int absfy2 = pIp->abs[1];
    int absfz3 = pIp->abs[2];

    if (pIp->abs[0] < 0){ absfx1 = -pIp->abs[0];}
    if (pIp->abs[1] < 0){ absfy2 = -pIp->abs[1];}
    if (pIp->abs[2] < 0){ absfz3 = -pIp->abs[2];}

    if (absfx1 > absfy2){ pIp->tmax = absfx1;}
    else{ pIp->tmax = absfy2; }

    if (absfz3 > pIp->tmax){pIp->tmax = absfz3;}

    *(pIp->ret) = pIp->tmax;
}

/*@
requires \valid(pIp);
requires \valid(pIp->abs+(0..2));
requires \valid(pIp->ret);
requires \separated(pIp, pIp->ret);
*/
void main1(TripleAbsMax *pIp)
{
    pIp -> abs[0] = 1;
    pIp -> abs[1] = 2;
    pIp -> abs[2] = -3;

    TripleAbsMaxFun(pIp);

    /*@ assert pIp -> tmax == 3; */
}

```

Fig. 9. A loop-free program with pointer aliasing and complex branching

architectural mismatch exists as SpecGen targets different programming and specification languages (Java/JML in SpecGen vs. C/ACSL in our work), creating translation barriers that make direct, automated comparison infeasible. Second, SpecGen was evaluated on a custom dataset rather than the established AutoSpec benchmark used in our study and other baseline works. By grounding our evaluation in widely recognized datasets while adding more challenging cases, we ensure that our results are both comparable to previous research and indicative of performance on complex softwares.

IX. RELATED WORK

a) Traditional approach: Traditional techniques for program specification and invariant inference can be broadly categorized into dynamic and static methods. Dynamic approaches infer specifications and invariants by analyzing runtime data collected during test execution. Daikon [9] executes test cases to gather program traces and produces a set of candidate

invariants based on a predefined library of templates, which are subsequently filtered using the observed data. Sharma et al. [24] employ a similar approach to generate candidate invariants, but additionally verify their soundness using SMT solvers. In contrast, DIG [25] bypasses template-based filtering and instead directly infers invariants from execution traces by integrating techniques from equation solving, polyhedra construction, and theorem proving. Despite their practicality, dynamic methods are highly dependent on the coverage and quality of the test suites.

Static methods eliminate the dependency on test cases and explore a variety of techniques, including constraint solving [1], [2], abstract interpretation [26] [27], abductive inference [22], Craig interpolation [3]–[5], recurrence analysis [28]–[30], shape analysis, and other methods [31] [32]. Colón *et al.* [1] present the first complete invariant generation method based on Farkas’ lemma and solves the non-linear invariant constraints via quantifier elimination, which is of dou-

ble exponential time complexity. Some heuristics are proposed to promote the efficiency of solving the constraints [2]. Cousot *et al.* [27] utilize convex polyhedra along with transformations and widening operations to discover linear invariants based on abstract interpretation. However, due to the intrinsic nature of abstract interpretation, the efficiency of the analysis heavily depends on the choice of abstract domains, and moreover, the approach offers no theoretical guarantee on the accuracy of the generated invariants due to the precision loss in abstraction. Lin *et al.* [3] synthesize inductive invariants using Craig interpolants derived from reachability analyses, and nonlinear Craig interpolant generation was first studied in [5]. The recurrence-based methods [28]–[30] derive recurrence relations from loops and solve these recurrences to obtain closed-form solutions as invariants. A key limitation of this approach, however, is restricted to the cases where the recurrences admit closed-form solutions or satisfy some strong conditions. Wu *et al.* [33] solve the strong and weak invariant synthesis problems of polynomial programs by combining invariant templates, semidefinite programming and constraint solving techniques. However, these traditional static methods are often not scalable and may require manual intervention. In contrast to these techniques, which are powerful yet often specialized for programs adhering to specific forms (e.g., certain types of loops or invariants), our approach leverages LLMs to achieve greater generality. In particular, some of these works focus primarily on theoretical challenges, and as a result, often exhibit limited efficiency and are difficult to apply directly to invariant generation at the program scale.

b) Machine learning and SMT-based approach:

Machine learning techniques and Large Language Models have been increasingly used for synthesizing loop invariants. Recent advancements [10], [11], [34]–[36], particularly in deep learning and reinforcement learning, have spurred new approaches for invariant and specification generation. CODE2INV [10], based on Counterexample-Guided Inductive Synthesis (CEGIS) [37], utilizes graph neural networks to learn neural representations of program structures. While CODE2INV employs a reinforcement learning approach, it often faces challenges with limited performance and sparse rewards. To address these issues, LIPuS [11] establishes a clearer division of labor between the capabilities of RL and SMT solvers and introduces a novel two-dimensional reward mechanism to alleviate the sparse reward problem of CODE2INV. Clause2Inv [12] refines the guess-and-check paradigm into a generate-combine-check framework, employing an LLM to generate candidate clauses and a counterexample-driven combiner to synthesize them into loop invariants. However, these approaches are mostly confined to numeric programs and do not constitute an end-to-end invariant generation framework; instead, they produce invariants in SMT format. Moreover, they are goal-directed, i.e., they rely on pre-given verification goals to generate accurate specifications. Empirical comparisons on numeric benchmarks show that our tool achieves competitive performance, and in some cases even outperforms these methods.

c) LLM-based approach: Pei *et al.* [38] demonstrates that fine-tuning pre-trained LLMs on code can effectively pre-

dict program invariants, with better performance than Daikon if the set of available tests is limited. However, same as Daikon, it has no formal guarantee for the validity of the generated invariants through formal verification. SpecGen [15] is a Java-oriented tool that employs a two-phase process to generate specifications: it first leverages LLM to produce invariant candidates, and then applies mutation operators with a weighted heuristic strategy to generate verifiable specifications when initial generation fails. Preguss [16] adopts a divide-and-conquer strategy that leverages potential runtime error (RTE) assertions from abstract interpretation-based static analyzers to guide LLM-based synthesis of minimal contracts, particularly preconditions that represent the weakest constraints sufficient to guarantee RTE-freeness. AutoSpec [13] decomposes programs into smaller parts and uses carefully designed prompts including few-shot examples to guide an LLM in generating specifications, which are then verified using Frama-C. AC-Inv [14] utilizes static analysis to extract essential information about loops and data structures, which is then embedded into prompts for an LLM to generate candidate invariants and abstract predicates. We provide a detailed comparison in Section VI with ACInv [14] and AutoSpec [13], both of which target C programs.

X. CONCLUSION

In this paper, we introduce a novel end-to-end framework for the automatic generation of C program specifications, including loop invariants and pre-/postconditions, by integrating symbolic execution, large language models (LLMs), and formal verification. Experimental results across diverse benchmarks demonstrate that our approach significantly outperforms existing methods in both invariant and functional specification generation. Furthermore, with the help of flattened memory model and inductive predicates, our tool supports programs with complex data structures including structs, pointers and recursively defined types (e.g., single linked lists), as well as inductive operations. Notably, even without explicit verification goals, our method maintains competitive accuracy and consistently surpasses prior tools.

For future work, we aim to extend the framework to more complex recursive data structures and recursive functions, and to strengthen invariant generation for challenging cases such as nested loops, where correlations among multiple induction variables arise. In particular, we plan to enhance the current LLM-based approach by integrating it with symbolic sampling and targeted training strategies to improve precision of the invariant generation of nested loops.

XI. DATA AVAILABILITY

All datasets, implementation code, experimental results, and supplementary materials including the appendix related to this work are publicly available at <https://github.com/anon-hiktyq/TSE2026-SESpec>.

REFERENCES

- [1] M. Colón, S. Sankaranarayanan, and H. Sipma, “Linear invariant generation using non-linear constraint solving,” in *Computer Aided Verification CAV*, ser. Lecture Notes in Computer Science, vol. 2725. Springer, 2003, pp. 420–432.

- [2] H. Liu, H. Fu, Z. Yu, J. Song, and G. Li, "Scalable linear invariant generation with farkas' lemma," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 204–232, 2022.
- [3] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanán, and H. Hansen, "FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 793–803.
- [4] T. Gan, L. Dai, B. Xia, N. Zhan, D. Kapur, and M. Chen, "Interpolant synthesis for quadratic polynomial inequalities and combination with EUF," in *Automated Reasoning - 8th International Joint Conference, IJCAR*, ser. Lecture Notes in Computer Science, vol. 9706. Springer, 2016, pp. 195–212.
- [5] T. Gan, B. Xia, B. Xue, N. Zhan, and L. Dai, "Nonlinear Craig Interpolant Generation," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 415–438.
- [6] D. Beyer, M. Kettl, and T. Lemberger, "Decomposing software verification using distributed summary synthesis," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1307–1329, 2024.
- [7] D. Wonisch and H. Wehrheim, "Predicate analysis with block-abstraction memoization," in *International Conference on Formal Engineering Methods*. Springer, 2012, pp. 332–347.
- [8] C. Calcagno, D. Distefano, P. O'Hearn, and H. Yang, "Compositional shape analysis by means of bi-abduction," in *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 2009, pp. 289–300.
- [9] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Sci. Comput. Program.*, vol. 69, no. 1, pp. 35–45, 2007, special issue on Experimental Software and Toolkits.
- [10] X. Si, A. Naik, H. Dai, M. Naik, and L. Song, "Code2Inv: A Deep Learning Framework for Program Verification," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 151–164.
- [11] S. Yu, T. Wang, and J. Wang, "Loop Invariant Inference through SMT Solving Enhanced Reinforcement Learning," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. Association for Computing Machinery, 2023, p. 175–187.
- [12] W. Cao, G. Wu, T. Xu, Y. Yao, H. Wei, T. Chen, and X. Ma, "Clause2inv: A generate-combine-check framework for loop invariant inference," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1009–1030, 2025.
- [13] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, and C. Tian, "Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification," in *Computer Aided Verification: 36th International Conference, CAV 2024, Montreal*. Springer-Verlag, 2024, p. 302–328.
- [14] R. Liu, G. Li, M. Chen, L.-I. Wu, and J. Ke, "Enhancing Automated Loop Invariant Generation for Complex Programs with Large Language Models," *CoRR*, vol. abs/2412.10483, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2412.10483>
- [15] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "SpecGen: Automated Generation of Formal Program Specifications via Large Language Models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, May 2025, pp. 666–666.
- [16] Z. Wang, T. Lin, M. Chen, H. Li, M. Yang, X. Yi, S. Qin, Y. Luo, X. Li, B. Gu, L. Lu, and J. Yin, "A tale of 1001 loc: Potential runtime error-guided specification synthesis for verifying large-scale programs," 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:284351244>
- [17] L. Zhou, J. Qin, Q. Wang, A. W. Appel, and Q. Cao, "VST-A: A foundationally sound annotation verifier," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, 2024.
- [18] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, N. Zhan, Z. Hu, and Q. Cao, "Qcp: A practical separation logic-based c program verification tool," 2025. [Online]. Available: <https://arxiv.org/abs/2505.12878>
- [19] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, "Frama-C: A software analysis perspective," in *International conference on software engineering and formal methods*. Springer, 2012, pp. 233–247.
- [20] C. Cadar, D. Dunbar, and D. R. Engler, "KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs," in *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008, pp. 209–224.
- [21] R. Alur, D. Fisman, S. Padhi, R. Singh, and A. Udupa, "Sygus-comp 2018: Results and analysis," *CoRR*, vol. abs/1904.07146, 2019.
- [22] I. Dillig, T. Dillig, B. Li, and K. McMillan, "Inductive invariant generation via abductive inference," in *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, ser. OOPSLA '13. Association for Computing Machinery, 2013, p. 443–456.
- [23] C. Liu, X. Wu, Y. Feng, Q. Cao, and J. Yan, "Towards general loop invariant generation: a benchmark of programs with memory manipulation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 129 120–129 145, 2024.
- [24] R. Sharma, S. Gupta, B. Hariharan, A. Aiken, P. Liang, and A. V. Nori, "A data driven approach for algebraic loop invariants," in *Programming Languages and Systems*. Springer Berlin Heidelberg, 2013, pp. 574–592.
- [25] T. Nguyen, D. Kapur, W. Weimer, and S. Forrest, "DIG: A Dynamic Invariant Generator for Polynomial and Array Invariants," *ACM Trans. Softw. Eng. Methodol.*, vol. 23, no. 4, pp. 1–30, 2014.
- [26] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fix-points," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '77. Association for Computing Machinery, 1977, p. 238–252.
- [27] P. Cousot and N. Halbwachs, "Automatic discovery of linear restraints among variables of a program," in *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '78. Association for Computing Machinery, 1978, p. 84–96.
- [28] E. Rodríguez-Carbonell and D. Kapur, "Automatic generation of polynomial loop invariants: Algebraic foundations," in *International symposium on Symbolic and algebraic computation (ISSAC)*. Springer, 2004, pp. 266–273.
- [29] —, "Generating all polynomial invariants in simple loops," *J. Symb. Comput.*, vol. 42, no. 4, pp. 443–476, 2007.
- [30] J. Cyphert and Z. Kincaid, "Solvable polynomial ideals: The ideal reflection for program analysis," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, pp. 724–752, 2024.
- [31] M. Chalupa and J. Strejček, "Backward Symbolic Execution with Loop Folding," in *Static Analysis*. Springer International Publishing, 2021, pp. 49–76.
- [32] T. Nguyen, M. B. Dwyer, and W. Visser, "SymInfer: inferring program invariants using symbolic states," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '17. IEEE Press, 2017, p. 804–814.
- [33] H. Wu, Q. Wang, B. Xue, N. Zhan, L. Zhi, and Z. Yang, "Synthesizing Invariants for Polynomial Programs by Semidefinite Programming," *ACM Trans. Program. Lang. Syst.*, vol. 47, no. 1, pp. 1 – 35, Feb. 2025.
- [34] S. Chakraborty, S. Lahiri, S. Fakhoury, A. Lal, M. Musuvathi, A. Rastogi, A. Senthilnathan, R. Sharma, and N. Swamy, "Ranking LLM-generated loop invariants for program verification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 9164–9175. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.614/>
- [35] H. Wu, C. Barrett, and N. Narodytska, "Lemur: Integrating Large Language Models in Automated Program Verification," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=Q3YaCghZNt>
- [36] M. A. A. Pirzada, G. Reger, A. Bhayat, and L. C. Cordeiro, "LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. Association for Computing Machinery, 2024, p. 1395–1407.
- [37] A. Abate, C. David, P. Kesseli, D. Kroening, and E. Polgreen, "Counterexample Guided Inductive Synthesis Modulo Theories," in *Computer Aided Verification*. Springer International Publishing, 2018, pp. 270–288.
- [38] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, "Can large language models reason about program invariants?" in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML '23. JMLR.org, 2023, pp. 27 496 – 27 520.